

Access control vulnerabilities

LAB	APPRENTICE Unprotected admin functionality →	✓ Solved
LAB	APPRENTICE Unprotected admin functionality with unpredictable URL →	✓ Solved
LAB	APPRENTICE User role controlled by request parameter →	✓ Solved
LAB	APPRENTICE User role can be modified in user profile →	✓ Solved
LAB	APPRENTICE User ID controlled by request parameter →	✓ Solved
LAB	APPRENTICE User ID controlled by request parameter, with unpredictable user IDs →	✓ Solved
LAB	APPRENTICE User ID controlled by request parameter with data leakage in redirect →	✓ Solved
LAB	APPRENTICE User ID controlled by request parameter with password disclosure →	✓ Solved
LAB	APPRENTICE Insecure direct object references →	✓ Solved
LAB	PRACTITIONER URL-based access control can be circumvented →	✓ Solved
LAB	PRACTITIONER Method-based access control can be circumvented →	✓ Solved
LAB	PRACTITIONER Multi-step process with no access control on one step →	✓ Solved
LAB	PRACTITIONER Referer-based access control →	✓ Solved

SUMMARY

Authentication confirms that the user is who they say they are.

Session management identifies which subsequent HTTP requests are being made by that same user.

Access control determines whether the user is allowed to carry out the action that they are attempting to perform.

vertical access control - restrict access to sensitive functionality to specific types of users.

horizontal access control - restrict access to resources to specific users.

context-dependent access control - restrict access to functionality and resources based upon the state of the application or the user's interaction with it

Access control vulnerabilities

check robots.txt

and make sure to check whether paths disclosed in Disallow has protection or not

Go through page source code of webapplication to find any urls

intercept the login requests and check for any user controlled role parameters like isadmin,admin.

Gather requests of testing the different features of website.

like change-email, forgot password, update information

deeply inspect for any roleid parameters

login using provided credentials

and check for url query string parameter manipulation [HPE]

GUID - Globally unique identifier (128 bit number)

used to uniquely identify an object in a computer system or network.

?userid=mimZbIPNsl9rRfhKICGX1gGD9CN9VLtJ

check if webapps using guid's and make sure if proper access control checks are implemented or not otherwise leads to

[HPE]

check for poorly implemented authorization checks manipulate url parameters

back-end application is built on a framework that supports the X-Original-URL header.

Some application frameworks support various non-standard HTTP headers that can be used to override the URL in the original request, such as X-Original-URL and X-Rewrite-URL.

If a website uses rigorous front-end controls to restrict access based on the URL, but the application allows the URL to be overridden via a request header then it might be possible to bypass the access controls

using a request like the following:

[Take image from screen shots]

POSTX is NOT an HTTP method.

It is a Postman “execution scope/event name” used inside Postman Test Scripts.

In Postman, there are different script execution events:

Name Meaning

prerequest Runs before the request is sent

test Runs after the response is received

post A legacy name sometimes used to refer to “after request” scripts

postx An extended post-execution event used internally by some Postman runtimes/plugins

POSTX is an internal/extended event that runs after the request (POST-event), but with more context available, such as response info, request variables, cookies

check Referrer: header and try to modify in accordance to overall request header.

access controls based on the user’s geographical location.

these access controls can be circumvented by using web proxies,vpn.

Developer's mistakes to correct

Provide server-side authorization to sensitive paths of robots.txt.

Never expose sensitive internal endpoints in HTML,JS,Comments

Trusting user-controlled role fields (role=isAdmin, roleId)

IDOR due to missing authorization on object identifiers (IDs, GUIDs)

Using URL-based access control (Front-end only checks)

[Authorization is tied to session, not URL.]

Improper handling of non-standard headers (X-Original-URL, X-Rewrite-URL)

Access control enforced only in the frontend (not backend)

Weak or missing server-side checks in update endpoints(Endpoints like change-email, edit-profile, forgot-password don’t verify the user.)

Relying on Referer header for authorization

Problem: Referer can be spoofed.

Geo-based checks without backend enforcement

Preventions

Never rely on obfuscation alone for access control.

Unless a resource is intended to be publicly accessible, deny access by default.

Wherever possible, use a single application-wide mechanism for enforcing access controls.

At the code level, make it mandatory for developers to declare the access that is allowed for each resource, and deny access by default.

Thoroughly audit and test access controls to ensure they work as designed.